ML Workflow for TH + Failure Gallery

Validation, leakage, metrics, and domain-aware practice

A model can look accurate and still be invalid.

What this lecture should make you able to do

By the end, you should be able to look at an ML result for TH data and ask the right credibility questions.

1 **Frame the workflow**	engineering problem → target → features → split → pipeline → model → diagnostics
2 **Explain leakage**	why using information unavailable at deployment invalidates evaluation
3 **Choose metrics**	why MAE, RMSE, R^2 and plots answer different questions
4 **Judge validation**	why random split, source-aware split, and OOD tests measure different things
5 **Recognise failures**	too-good-to-be-true metrics, source bias, extrapolation, overfitting, miscalibrated uncertainty

Why workflow matters?

TH data is rarely just independent rows in a spreadsheet. It usually carries physics, source, regime, simulator, and measurement structure.

Same facility / campaign	hidden correlations across rows
Same simulator / code version	model may learn code signatures
Same geometry / scaling	source-specific behaviour
Regime boundaries	interpolation in one regime, extrapolation in another
Sparse safety regions	rare but important failure/margin cases

Standard ML assumption

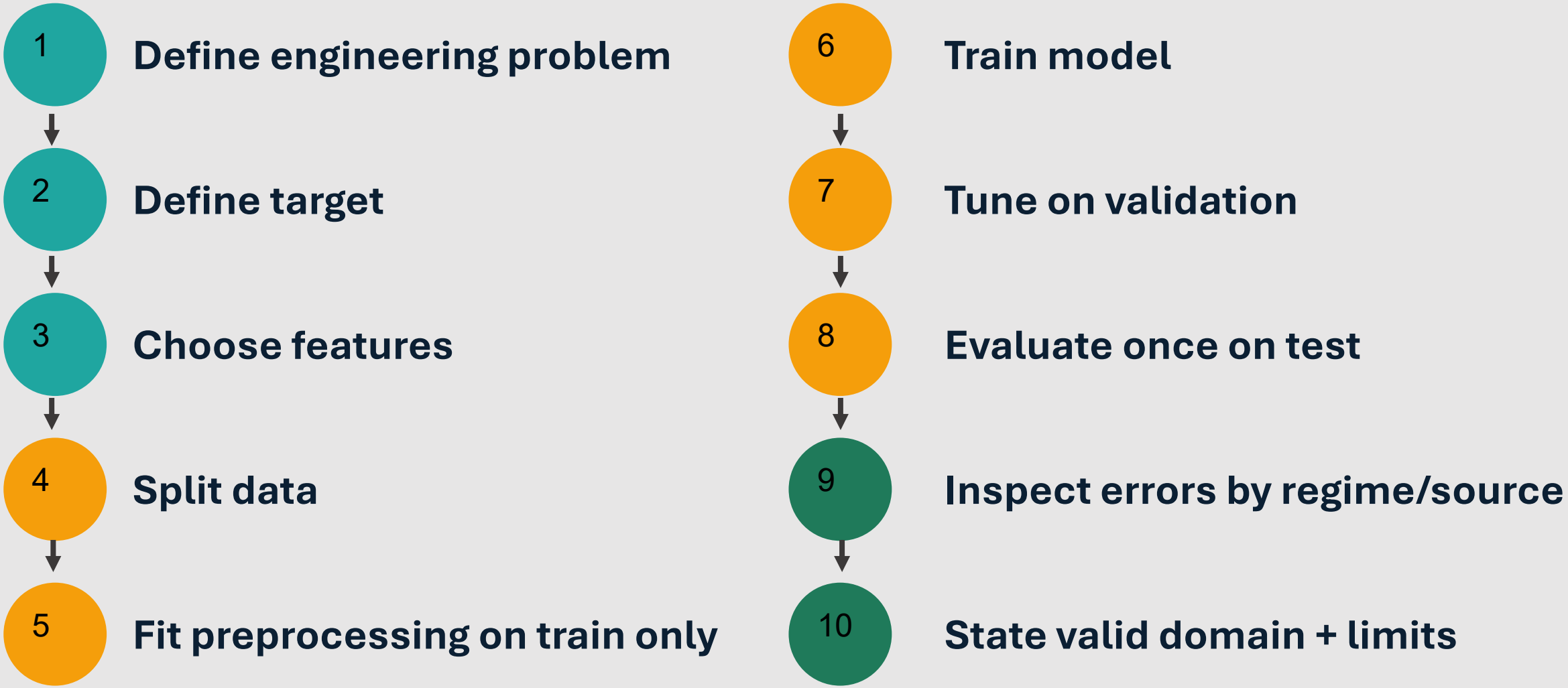
Rows are representative samples from the same deployment distribution.

NTH reality

Data often has source structure, scaling distortion, and out-of-domain risk.

Result: validation design matters as much as algorithm choice.

A typical supervised-learning workflow



Step 1–2: from engineering problem to target

The target is not just a column name. It is the engineering quantity, time horizon, and use context.

CHF prediction target: CHF value	**regression**
DNBR surveillance target: DNBR margin / threshold	**Regression/ classification**
LOCA diagnosis target: break class or size	**Classification/regression**
Code emulator target: PCT, pressure peak, void fraction	**surrogate regression**
Sensor validation target: healthy expected sensor value	**soft-sensor regression**

Ask before modelling:

- **What quantity do we need?**
- **At what time or condition?**
- **For what decision?**

Step 3: feature definition is an engineering decision

Features must be available at prediction time and physically meaningful for the intended use.

Operating conditions
pressure, mass flux, inlet temperature,

Geometry
diameter, heated length, spacer, channel

State descriptors
quality, subcooling, void fraction, regime

Material / fluid properties
density, viscosity, saturation properties

Metadata
facility/source, simulator version, campaign ID

1. **Available?** — Would this be known before deployment?
2. **Not target-derived?** — Does it leak future or output information?
3. **Redundant?** — Do variables encode the same thermodynamic state?
4. **Stable?** — Will definition and units match across sources?
5. **Metadata use?** — Input feature, grouping variable, or diagnostic label?

Step 4: train / validation / test logic

The split is not bookkeeping. It controls what performance claim you are allowed to make.

Train	Validation	Test
Used for: fit model parameters and preprocessing	**Used for:** tune hyperparameters, choose features, compare models	**Used for:** one independent check after decisions are frozen
Should not be used for: **not final performance**	**Should not be used for:** **not final claim**	**Should not be used for:** **not a playground**

If you keep using the test set to improve the model, it becomes another validation set.

Split design depends on the engineering question.

Different splits answer different questions. Choose the split that matches the intended use.

Random row split

- Can the model interpolate among similar rows?
- Good for first baseline; often optimistic for source-structured data

Source-grouped split

- Can it generalise to a new facility, campaign, or database source?
- More realistic for CHF and experimental databases

Regime-aware split

- Can it handle a new pressure, quality, mass-flux, or flow-regime region?
- Tests domain-of-applicability and extrapolation risk

Time-based split

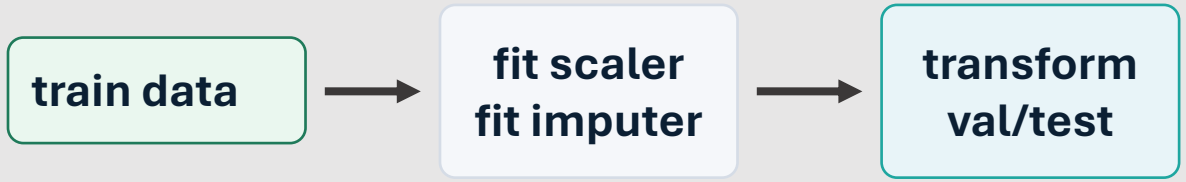
- Can it forecast future operation from past data?
- Important for monitoring, sensor drift, and transient data

Teaching rule: do not call a result “general” if the split only tested interpolation among similar cases.

Step 5: preprocessing must be fitted on training data only

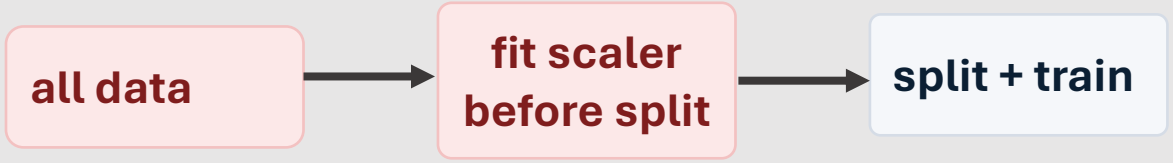
Scaling, imputation, feature selection, and transformations can leak test information if fitted too early.

Correct



The validation/test distributions remain unseen during fitting.

Leakage



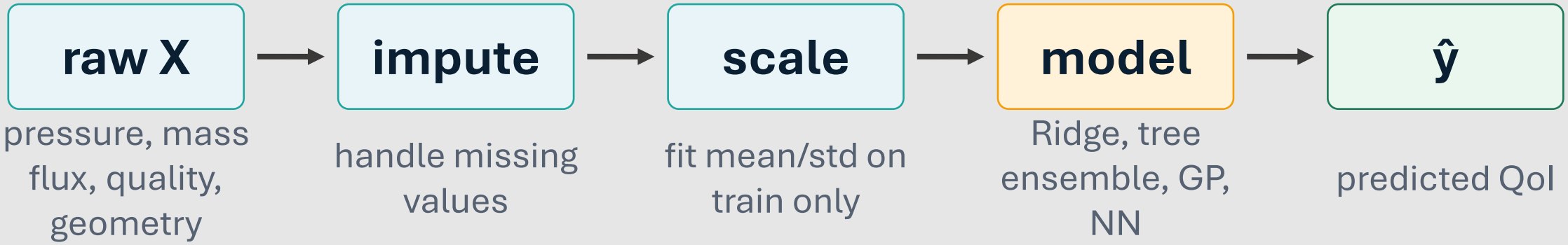
Test-set statistics have influenced the model development.

Conceptual solution:

Use a pipeline: preprocessing and model are fitted together on training data.

The pipeline idea: make the safe path easy

A pipeline bundles all transformations with the model so the same fitted operations are applied consistently.

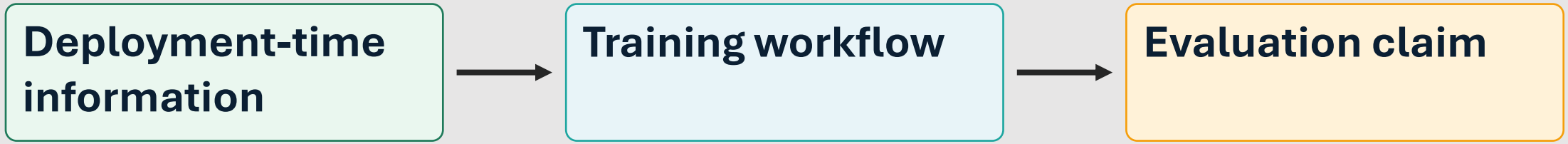


Train call fits every stage using training data.
Validation/test calls reuse fitted stages only.

This is why the Day 1 lab uses a Pipeline even for simple models.

Leakage: the central workflow failure

Leakage means the model or evaluation uses information that would not be available in real deployment.



Feature leakage:
target-derived variables, future values, post-event information

Preprocessing leakage:
fit scaler/imputer/feature selector on all data

Split leakage:
duplicates, same source, same simulator traces across train/test

Test-set leakage:
repeated tuning until the test score looks good

Failure mode: target-derived or future information

A model can become excellent by learning a shortcut that would not exist at prediction time.

Example:
Predict CHF but include a variable derived from measured CHF, measured dryout state, or post-test classification.

Why it looks good:
The model sees a proxy for the answer, so test metrics can become unrealistically strong.

How to prevent it:
Create a "prediction-time available" checklist for every feature before modelling.

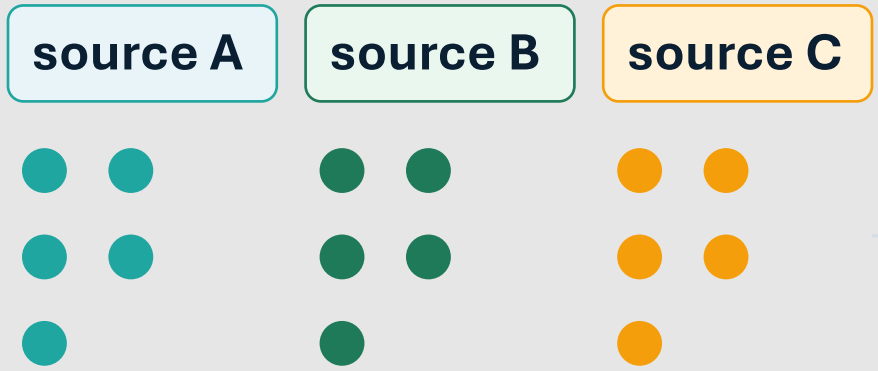
Allowed features
pressure
mass flux
quality
geometry

Forbidden shortcuts
post-test labels
target-derived ratios
future sensor values

Question: would this feature be known before making the prediction?

Failure mode: random split across related sources

Random row splits can mix near-duplicate cases, same facility effects, and same simulator artefacts across train and test.



Naive random split
Train and test contain A, B, and C. Test score measures interpolation within known sources.

Source-aware split
Train on A+B, test on C. Score measures transfer to a held-out source.

For CHF and experimental TH databases, source-aware validation is often more credible than a row-wise split.

Step 8: choose metrics that match the engineering question

For regression, use a small set of complementary metrics and interpret them in units.

MAE

mean absolute error

Good for:
typical absolute error; easy to explain in engineering units

Caution:
less sensitive to rare large misses

RMSE

root mean square error

Good for:
penalises large errors; useful when large misses matter

Caution:
can be dominated by outliers

R^2

variance explained

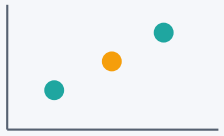
Good for:
quick relative score; familiar to many users

Caution:
can look good while safety-relevant regions fail

Never present a metric table without diagnostic plots and regime/source breakdowns.

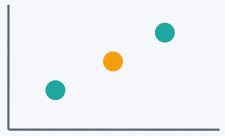
Step 9: diagnostic plots reveal what metrics hide

Plots connect model error back to physics, regimes, sources, and deployment limits.



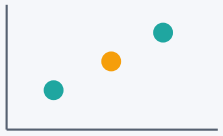
Parity plot

Do predictions follow observations?



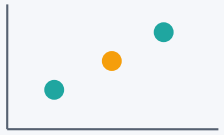
Residual vs predicted

Bias, heteroscedasticity, large misses



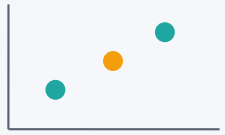
Residual vs pressure

Pressure-range dependence



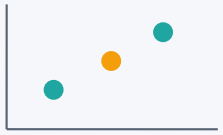
Residual vs mass flux

Flow-condition dependence



Residual vs quality

Two-phase / dryout regime dependence



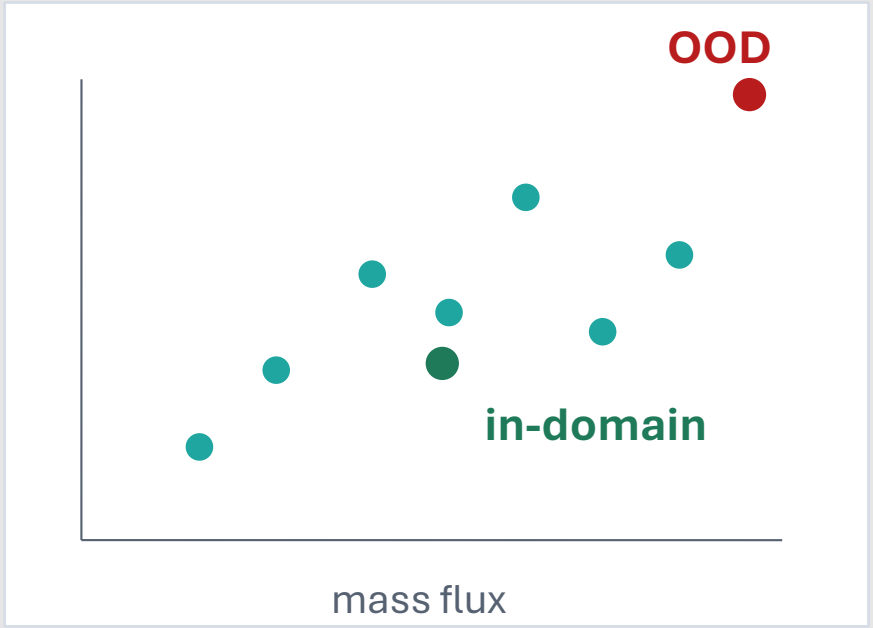
Error by source

Facility or database bias

Diagnostic habit: ask “where does the model fail?” before asking “which model wins?”

Domain-aware validation: in-domain is not out-of-domain

A test case can be statistically held out and still be physically close to training data — or physically outside it.



In-domain use
Similar sampled ranges, regimes, geometry, sources and measurement definitions. Errors are more likely interpolation-like.

Out-of-domain use:
Different pressure/quality range, facility, scaling, sensor behaviour, simulator version, or physical regime. Generalisation is uncertain.

Domain statement = what the model is allowed to be used for.

Failure gallery: five ways to fool yourself

The following failures are compact, memorable examples.
Each can produce impressive-looking numbers.

1. **Data leakage:** model sees forbidden information
2. **Bad split:** test set is not independent in engineering sense
3. **Wild extrapolation:** model used outside its valida envelope
4. **Overfit model:** training error improves while validation fails
5. **Miscalibrated uncertainty:** intervals do not cover what they claim

Failure-gallery question: which workflow step was broken?

Failure 1 — Data leakage: too good to be true

Leakage often appears as a surprisingly strong test score with little physical explanation.

What happened?
A target-derived variable, future sensor value, all-data preprocessing, duplicate, or source clue entered the training process.

What it looks like?
Very high R^2, tiny MAE/RMSE, near-perfect parity plot, weak dependence on algorithm choice.

What to do?
Audit features, move split before preprocessing, remove duplicates, keep test untouched, rerun with grouped validation.

Suspicious metric pattern	
Train R^2	0.999
Val R^2	0.998
Test R^2	0.998
Physics explanation	unclear

Rule: unusually good performance deserves suspicion, not celebration.

Failure 2 — Bad split: over-optimistic validation

A split can be formally random and still not test the intended generalisation question.

Naive row split:

- same facility in train and test
- near-duplicate operating points
- same simulator/correlation signatures
- performance looks excellent

→

Source-aware or regime-aware split:

- hold out facility/source/regime
- tests transfer rather than interpolation
- performance may drop
- claim becomes more honest

Which result is better?
The one that matches the deployment question

Failure 3 — Wild extrapolation: confident outside the map

Most supervised models interpolate better than they extrapolate. This is especially important near safety limits.

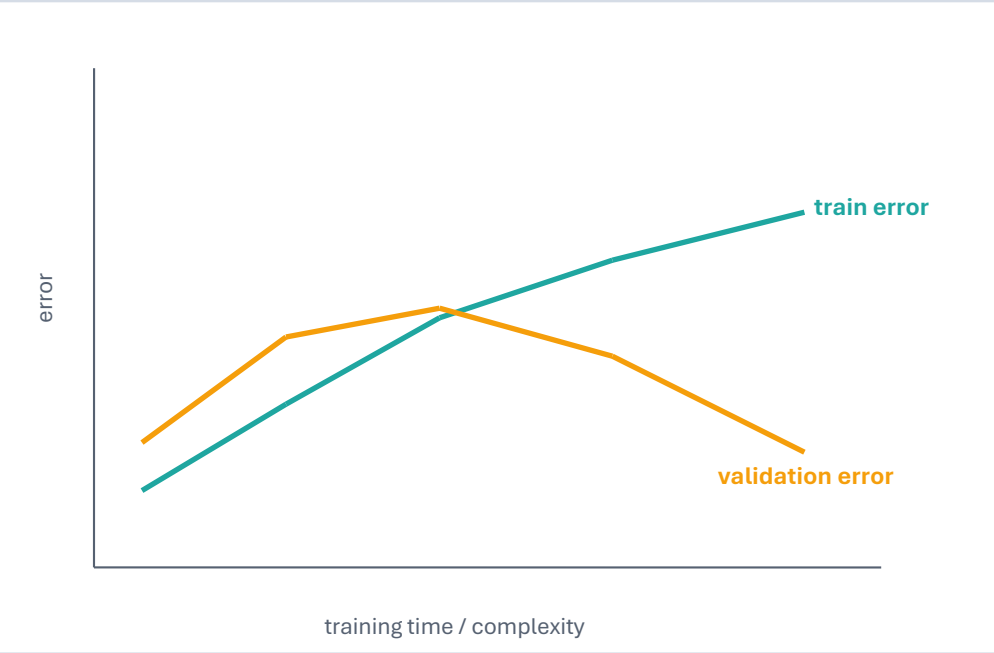
Inputs	train	request
pressure	10–16 MPa	7 MPa
mass flux	1000–4000 kg/m^{2}s	5200 kg/m^{2}s
quality	-0.1–0.4	0.7
diameter	8–12 mm	5 mm

Bad behaviour:
The model gives a point prediction with no warning, even though input conditions are outside the training envelope.

Good behaviour:
Flag OOD inputs, widen uncertainty, request high-fidelity analysis, or fall back to conservative rules.

Failure 4 — Overfitting: better training fit, worse generalisation

Flexible models can fit training data extremely well while failing on validation or held-out sources.



What happened?
The model captured noise, source-specific quirks, or high-dimensional shortcuts instead of robust TH behaviour.

Controls:
Use baselines, validation curves, regularisation, early stopping, simpler models, and grouped validation.

More parameters ≠ more credibility.

Failure 5 — Miscalibrated uncertainty

Uncertainty estimates must be checked. Reporting intervals is not the same as being uncertain correctly.

Claim → **"90% prediction intervals"** → **Check**

Do about 90% of observations fall inside?

Under-confident: intervals too wide; model may be conservative but less useful

Well-calibrated: nominal coverage roughly matches empirical coverage

Over-confident: intervals too narrow; dangerous near safety limits

Cross-cutting failure: the simulator trap

A model trained on simulator data may become very good at reproducing the simulator, not the plant or experiment.

RELAP / TRACE / MAAP / MELCOR or CFD-generated data → **ML model learns simulator signatures** → **Plant / facility behaviour under shift**

facility-to-plant scaling:
geometry, power, flow regime, boundary conditions

sensor drift / noise
real sensors differ from clean simulator signals

control logic differences:
operator actions and protection systems differ

model-form bias
simulator closure errors can be learned as truth

Simulator-trained ML should state its validation ladder and fallback behaviour.

What reviewers should ask before trusting an ML result

These questions convert workflow discipline into an evidence checklist.

Problem:
What engineering decision does the model support ?

Data:
Where did the data come from? Are sources documented?

Target/features:
Are features available at prediction time?

Split:
Does the split match the deployment question?

Preprocessing:
Was every transform fitted on train only?

Metrics:
Are metrics and plots reported by regime/source?

OOD:
What happens outside the validated envelope?

Uncertainty:
Are uncertainty intervals calibrated?

Fallback:
What safe action occurs when the model should not be trusted?

If these are unclear, the algorithm choice is premature.

Take-home messages

1. A supervised ML workflow starts with the engineering question, target, and features.
2. Train / validation / test split logic controls what performance claim is valid.
3. Leakage can make a model look accurate while invalidating the evaluation.
4. Metrics must be paired with diagnostic plots and regime/source error analysis.
5. In-domain use and out-of-domain use are different engineering claims.

Bridge to lab: build a CHF model, then try to break your own validation claim.

Pre-lab bridge: the Day 1 CHF workflow

The lab will practise this workflow on a CHF regression problem.